F.E.A.R. Project — User Manual

Fully Encrypted Anonymous Routing

Version: 1.2 (v0.4.1) **Author:** Shchuchkin E. Yu.

Table of Contents

1. Introduction
2. Architecture and Security
3. System Requirements
4. Building the Project

5. Programs
6. Quick Start Scenarios
7. Detailed Usage
8. ECDH Key Exchange
9. Identity Verification
10. Audio Calls
11. Video Calls
12. File Transfer
13. Troubleshooting
14. FAQ

Introduction

F.E.A.R. (Fully Encrypted Anonymous Routing) is a cross-platform open-source messenger designed for maximum privacy and security.

Key Features

- **End-to-end encryption (E2EE):** All messages are encrypted on the sender's device and decrypted only on the recipient's device
- **Zero-knowledge server:** The server cannot access message content, keys, or decrypted data
- **ECDH key exchange:** Automatic key distribution via X25519 — no pre-shared secrets needed
- **Identity verification:** Ed25519 keypairs with Trust On First Use (TOFU) model
- **Encrypted voice calls:** Opus codec with AES-256-GCM over UDP
- **Encrypted video calls:** VP8 codec with AES-256-GCM, SDL3 display, adaptive bitrate
- **File transfer:** Encrypted file sharing with CRC32 integrity verification
- **Open source:** Full transparency for audit and verification

What's New in v0.4.1

ECDH Key Exchange:

- Automatic room key generation and distribution — no manual key sharing required
- CLI: `--create` (generate key) and `--join` (receive key via ECDH)
- GUI: Create Room / Join Room / Connect buttons
- X25519 ephemeral keypairs with Ed25519-signed responses (MITM protection)

Identity Verification:

- Each client generates a persistent Ed25519 keypair
- Peer public keys are saved on first contact (TOFU)
- Key mismatch on reconnection triggers a security warning

Android v0.4.1:

- Full feature parity: ECDH, identity, video calls, file transfer
- Light and dark theme toggle
- Push notifications for background messages

Architecture and Security

Security Model

F.E.A.R. uses a client-server architecture with end-to-end encryption:

```
[Client A] <——encrypted——> [Server] <——encrypted——> [Client B]
                                |
                    (cannot read content)
```

The server performs routing only and has **no access to:**

- Message content
- Room encryption keys
- Decrypted data

Cryptographic Algorithms

AES-256-GCM

Purpose: Encryption of all messages, audio, and video data

Parameter	Value
Key size	256 bits (32 bytes)
Mode	GCM (Galois/Counter Mode)
Nonce size	96 bits (12 bytes)
Auth tag	128 bits (16 bytes)

AES-256-GCM provides:

- **Confidentiality:** Data is unreadable without the key
- **Authentication:** Tampering is detected via the auth tag
- **Replay protection:** Each message uses a unique nonce

X25519 (ECDH Key Exchange)

Purpose: Secure room key distribution without pre-shared secrets

The ECDH protocol allows two parties to derive a shared secret over an insecure channel. Combined with Ed25519 signatures, it prevents man-in-the-middle attacks.

Ed25519

Purpose: Identity signing and verification

Each client maintains a persistent Ed25519 keypair. During ECDH key exchange, the room creator signs the response with their identity key. Peers can verify authenticity.

BLAKE2b (Key Derivation)

Purpose: Deriving audio and video subkeys from the room master key

- Audio subkey: `crypto_kdf(id=1, ctx="fearaudi", master_key)`
- Video subkey: `crypto_kdf(id=2, ctx="fearvide", master_key)`

CRC32

Purpose: File transfer integrity verification (non-cryptographic)

Protocol Format

Every TCP message follows this wire format:

```
[2 bytes: room_len]
[room name]
[2 bytes: name_len]
[sender name]
[2 bytes: nonce_len]
[nonce (12 bytes)]
[1 byte: message_type]
[4 bytes: ciphertext_len]
[ciphertext + auth_tag]
```

Message types:

Type	Name	Description
0	TEXT	Encrypted text message
1	FILE_START	File transfer metadata
2	FILE_CHUNK	File data chunk (8KB)
3	FILE_END	File transfer completion
4	USER_LIST	Room participants (service, zero nonce)
15	KEY_REQUEST	ECDH key request (service, zero nonce)
16	KEY_RESPONSE	ECDH key response (service, zero nonce)

System Requirements

Minimum Requirements

- **OS:** Windows 10/11 (64-bit) or Linux (Ubuntu 20.04+, Debian 11+)
- **CPU:** Intel Core i3 / AMD Ryzen 3 or equivalent
- **RAM:** 2 GB
- **Disk:** 200 MB free space

- **Network:** Internet or LAN connection

Build Requirements

- Git, CMake 3.15+, C++17 compiler (GCC 8+, MinGW-w64 8+)
- **Libraries:** libsodium, Qt 6.2+, PortAudio, Opus, FFmpeg, libvpx, SDL3

Building the Project

Linux

```
# Install dependencies (Ubuntu/Debian)
./build.sh deps

# Build
./build.sh
```

Windows

```
build.bat
```

See BUILD.md for detailed instructions, Windows library setup, and troubleshooting.

Build Output

```
build/
├── fear_gui          # GUI application
└── bin/
    ├── fear          # Console client/server
    ├── audio_call    # Voice call utility
    ├── video_call    # Video call utility
    ├── key-exchange  # Key exchange utility
    └── updater       # Update manager
```

Programs

1. fear (Console Client/Server)

Location: `build/bin/fear`

Generate Room Key

```
./fear genkey
```

Output:

```
z6aK3_k9I7rmpy6Sn-84QZ9Yc0p3T7VhzReWCKE0x4I
Room key generated successfully.
IMPORTANT: Copy the key above and share it securely.
           The key is NOT saved to disk for security reasons.
```

The key is output to stdout only. In the GUI, it is automatically copied to clipboard.

Start Server

```
./fear server --port 7777
```

The server requires no keys — it only relays encrypted data.

Connect as Client

Three connection modes are available:

Create Room (auto-generate key):

```
./fear client --host SERVER_IP --port 7777 \
    --room myroom --name Alice --create
```

Join Room (ECDH key exchange):

```
./fear client --host SERVER_IP --port 7777 \
    --room myroom --name Bob --join
```

Connect with known key (stdin — recommended):

```
echo "YOUR_KEY" | ./fear client --host SERVER_IP --port 7777 \
    --room myroom --name Charlie
```

Connect with key file:

```
./fear client --host SERVER_IP --port 7777 \
    --room myroom --name Charlie --key-file room_key.txt
```

Arguments:

Argument	Required	Description
`--host`	Yes	Server IP or hostname
`--port`	Yes	Server TCP port (1024-65535)
`--room`	Yes	Room name (1-255 chars)
`--name`	Yes	Your username (1-255 chars)
`--create`	No	Auto-generate key, create room
`--join`	No	Join via ECDH key exchange
`--key-file`	No	Read key from file

In-chat commands:

- Type text and press Enter to send a message
- `/sendfile path/to/file` to send a file
- Ctrl+C to exit

2. fear_gui (GUI Application)

Location: `build/fear_gui`

Launch:

```
cd build && ./fear_gui     # Linux
fear_gui.exe               # Windows
```

Connection modes:

- **Create Room:** Menu > Connection > Create Room — auto-generates key, copies to clipboard
- **Join Room:** Menu > Connection > Join Room — ECDH exchange, no key needed
- **Connect:** Menu > Connection > Connect — enter all fields manually

Features:

- Chat with message history
- User list panel
- File transfer button
- Audio calls: Menu > Audio call > Start audio call
- Video calls: Menu > Video call > Start video call
- Key exchange: Menu > Keys > Key exchange
- Update check: Menu > Help > Check for updates

3. key-exchange (Key Exchange Utility)

Location: `build/bin/key-exchange`

Interactive utility for Curve25519 (ECDH) key exchange:

```
=== F.E.A.R. Key Exchange ===
1. Generate key pair
2. Encrypt message
3. Decrypt message
4. Exit
```

Workflow:

1. Both users generate keypairs (option 1)
2. Exchange public keys over any channel
3. User A encrypts the room key with User B's public key (option 2)
4. User B decrypts the room key (option 3)

Public keys can be shared openly. The secret key must never be transmitted.

4. audio_call (Voice Calls)

Location: `build/bin/audio_call`

```
# Generate key
./audio_call genkey

# List audio devices
./audio_call listdevices

# Listen for incoming call
echo "KEY" | ./audio_call listen 50000

# Make a call
echo "KEY" | ./audio_call call REMOTE_IP 50000
```

Specs: Opus codec, 48 kHz, adaptive bitrate, AES-256-GCM encryption.

5. video_call (Video Calls)

Location: `build/bin/video_call`

```
# Generate key
./video_call genkey

# List cameras and audio devices
./video_call listdevices

# Listen for incoming call
echo "KEY" | ./video_call listen 50000
```

```
# Make a call
echo "KEY" | ./video_call call REMOTE_IP 50000

# Options
echo "KEY" | ./video_call call REMOTE_IP 50000 --quality high
echo "KEY" | ./video_call listen 50000 --no-camera
```

Options:

Option	Description
`--quality low|medium|high`	Video quality preset (default: medium)
`--camera DEVICE`	Camera device
`--no-camera`	Receive-only mode (no camera)
`--no-video`	Audio only
`--no-audio`	Video only
`--width N --height N`	Custom resolution
`--fps N`	Custom framerate
`--bitrate N`	Custom bitrate (kbps)

6. updater (Update Manager)

Location: `build/bin/updater`

```
./updater
```

Checks for new versions, downloads, and applies updates.

Quick Start Scenarios

Scenario 1: Two Users on LAN

Step 1: Alice generates a key and starts the server:

```
./fear genkey
# Output: z6aK3_k9I7rmpy6Sn-84QZ9Yc0p3T7VhzReWCKE0x4I

./fear server --port 7777
```

Step 2: Alice connects and creates a room:

```
./fear client --host 127.0.0.1 --port 7777 \
    --room myroom --name Alice --create
```

Step 3: Alice shares her IP (e.g., 192.168.1.10) and the key with Bob.

Step 4: Bob joins:

```
echo "z6aK3_k9I7rmpy6Sn-84QZ9Yc0p3T7VhzReWCKE0x4I" | \
    ./fear client --host 192.168.1.10 --port 7777 \
    --room myroom --name Bob
```

They can now chat securely.

Scenario 2: ECDH Key Exchange (No Pre-Shared Key)

Step 1: Alice starts the server and creates a room:

```
./fear server --port 7777
./fear client --host 127.0.0.1 --port 7777 \
    --room myroom --name Alice --create
```

Step 2: Alice tells Bob the server IP, port, and room name (no key needed).

Step 3: Bob joins via ECDH:

```
./fear client --host 192.168.1.10 --port 7777 \
    --room myroom --name Bob --join
```

The key exchange happens automatically. Bob receives the room key via encrypted ECDH channel.

Scenario 3: GUI Usage

1. Launch `fear_gui`
2. Menu > Connection > Create Room
3. Fill in server, port, room, name — click **Create**
4. Key is auto-generated and copied to clipboard
5. Share key with participant (or have them use **Join Room**)
6. Participant: Menu > Connection > Join Room — click **Join**
7. Chat, send files, start audio/video calls

ECDH Key Exchange

How It Works

When `--create` is used, the client generates a random room key. When another client connects with `--join`:

1. Joiner sends `KEY_REQUEST` with their ephemeral X25519 public key
2. Creator generates their own ephemeral X25519 keypair
3. Creator encrypts the room key using `crypto_box` (shared secret from X25519)
4. Creator signs the response with their Ed25519 identity key
5. Creator sends `KEY_RESPONSE` with encrypted key + signature
6. Joiner verifies the Ed25519 signature and decrypts the room key

Security guarantees:

- The room key never travels in plaintext
- Ed25519 signature prevents man-in-the-middle attacks
- Ephemeral keypairs provide forward secrecy for the exchange

---

Identity Verification

TOFU (Trust On First Use)

Each F.E.A.R. client generates a persistent Ed25519 keypair stored in `.fear/identity/`. When connecting to a peer for the first time, their public key is saved locally. On subsequent connections, the key is compared:

- **Match:** Connection proceeds normally
- **Mismatch:** Security warning — the peer's identity may have changed (or an attacker is present)

Trusted Keys Management

In the GUI, trusted keys can be managed via Menu > Keys > Trusted keys.

Audio Calls

Requirements

- Microphone and speakers/headphones
- Direct network connection between peers (or port forwarding)
- Low network latency (< 100ms recommended)

Setup via GUI

1. Menu > Audio call > Start audio call
2. Generate key (auto-copied to clipboard) or enter existing key
3. Select audio input/output devices
4. **Receiver:** Enter local port, click "Start Listening"
5. **Caller:** Enter remote IP and port, click "Start Call"

Setup via Console

Receiver:

```
./audio_call genkey
echo "KEY" | ./audio_call listen 50000
```

Caller:

```
echo "KEY" | ./audio_call call 192.168.1.100 50000
```

Both parties must use the same key.

Video Calls

Requirements

- Webcam (optional — "no camera" mode available)
- Microphone and speakers/headphones
- Direct network connection between peers

Quality Presets

Preset	Resolution	FPS	Bitrate
LOW	320x240	15	200 kbps
MEDIUM	640x480	25	500 kbps
HIGH	1280x720	30	1500 kbps

Setup via GUI

1. Menu > Video call > Start video call
2. Generate key or enter existing key
3. Select camera (or "No camera" for receive-only)
4. Choose quality preset
5. **Receiver:** Click "Start Listening"
6. **Caller:** Enter remote IP and port, click "Start Call"

Setup via Console

Receiver:

```
echo "KEY" | ./video_call listen 50000 --quality medium
```

Caller:

```
echo "KEY" | ./video_call call 192.168.1.100 50000 --quality medium
```

Features

- **Peer disconnect detection:** If no data for 5 seconds, a black frame is displayed
- **Auto reconnect:** When a peer reconnects, decoder and fragment buffer reset automatically
- **No camera mode:** Receive video without sending — use `--no-camera` or select in GUI

File Transfer

Sending a File

GUI: Click "Send file" button, select file.

Console:

```
/sendfile /path/to/file.pdf
```

Receiving a File

Files are automatically saved to the `Downloads/` directory.

Integrity Verification

F.E.A.R. verifies file integrity using CRC32 checksums. Corrupted files are automatically deleted.

Limitations

- Chunk size: 8192 bytes
- Files are encrypted with the room key
- No file size limit (bounded by available memory)

Troubleshooting

Connection Issues

"Connection refused":

- Verify the server is running
- Check IP address and port
- Check firewall settings

"Name already exists in room":

- Choose a different username
- Wait for the previous session to disconnect

Encryption Issues

Messages not decrypting:

- All participants must use the same room key
- Verify key was not corrupted during copy (should be 44 Base64 chars)
- Regenerate key and redistribute

Call Issues

No audio/video:

- Check microphone/camera permissions
- Verify correct device is selected
- Check firewall allows UDP traffic on the call port

High latency:

- Use wired connection instead of Wi-Fi
- Close bandwidth-intensive applications
- Check network latency: `ping REMOTE_IP`

Build Issues

See BUILD.md for detailed troubleshooting.

FAQ

Q: Is F.E.A.R. fully anonymous?

A: F.E.A.R. provides confidentiality of message content but does not hide the fact of communication. The server and network observers can see IP addresses, connection times, and data volumes. For full anonymity, use F.E.A.R. over VPN or Tor.

Q: Do I need to trust the server?

A: The server cannot read your messages (E2EE). However, it can see metadata (who communicates with whom, when). For maximum security, run your own server.

Q: How often should keys be changed?

A: Change the room key when adding or removing participants, and periodically for long-term rooms. With `--create` / `--join`, each room session can use a fresh key easily.

Q: What if a room key is compromised?

A: Generate a new key (`./fear genkey`), create a new room, and redistribute the key to trusted participants only.

Q: Does F.E.A.R. log messages?

A: No. Messages exist only in memory during the session. The server does not store messages.

Q: Can I use group chats?

A: Yes. Any number of users can join a room. All messages are encrypted with the shared room key.

Q: Is there a mobile app?

A: Yes. An Android client is available at fear-mobile. It is fully compatible with the desktop server and supports all features including ECDH, identity verification, audio/video calls, file transfer, themes, and push notifications.

License

F.E.A.R. Project is distributed under the **MIT License**.

Contact

GitHub: https://github.com/shchuchkin-pkims/fear **Issues:** https://github.com/shchuchkin-pkims/fear/issues

Reporting Bugs

1. Open a GitHub issue
2. Include: F.E.A.R. version, OS, steps to reproduce, expected vs actual behavior
3. Attach logs if available

Stay Anonymous. Stay Secure. Shchuchkin E. Yu. F.E.A.R. Project